

Got it — you want a **developer-first, dev-mode execution plan** (not high-level architecture), optimized for **local development using FAISS + SQLite + local storage**, while still keeping future production in mind.

I'll rewrite this as a **step-by-step implementation playbook** your engineers can directly follow.

Healthcare + Fitness Agent

Developer Execution Plan (DEV MODE – Local First)

1. **Goal of Dev Mode**

Build a **fully working offline MVP** that can:

- Ingest PDFs (health reports)
- Ask user questions (chat)
- Analyze health data
- Generate:
 - Risk insights
 - Fitness plans
 - Diet & lifestyle guidance

 All without cloud dependency (except LLM API)

2. **Local Architecture**

React / Web UI (Localhost)

↓

FastAPI Backend

↓

LangChain / LangGraph Orchestrator

↓

| FAISS (Vector DB) |
| SQLite (Structured DB) |
Local File Storage (PDFs)

↓

Azure OpenAI API

3. 📁 Project Structure (MANDATORY)

healthcare-agent/

```
|  
|  
| └─ backend/  
|   |  
|   | └─ app.py  
|   | └─ routes/  
|   | └─ services/  
|   | └─ agents/  
|   | └─ utils/  
|   | └─ db/  
|   | └─ models/  
|  
| └─ data/  
|   |  
|   | └─ pdfs/  
|   | └─ faiss_index/  
|
```

```
├── frontend/
|
├── config/
|   ├── settings.py
|
└── requirements.txt
```

4. 🗄️ SQLite Setup

Step 1: Initialize DB

```
import sqlite3
```

```
conn = sqlite3.connect("health.db")
```

```
cursor = conn.cursor()
```

Step 2: Tables

Users

```
CREATE TABLE users (
    user_id TEXT PRIMARY KEY,
    email TEXT,
    age INTEGER,
    gender TEXT,
    height REAL,
    weight REAL
);
```

Health Records

```
CREATE TABLE health_records (
```

```
id INTEGER PRIMARY KEY AUTOINCREMENT,  
user_id TEXT,  
parameter TEXT,  
value REAL,  
unit TEXT,  
reference_range TEXT,  
status TEXT  
);
```

5. PDF Ingestion Pipeline

Step-by-Step Flow

Upload PDF → Save Locally → Extract Text → Parse → Store → Embed

Implementation

Save File

```
file_path = f"data/pdfs/{user_id}_{filename}"  
  
with open(file_path, "wb") as f:  
    f.write(file.file.read())
```

Extract Text

Use:

- PyMuPDF OR pdfplumber

```
import fitz
```

```
doc = fitz.open(file_path)
```

```
text = ""
```

```
for page in doc:
```

```
    text += page.get_text()
```

Parse Medical Data (LLM Prompt)

```
prompt = """
```

```
Extract lab test values in JSON:
```

```
- test_name
```

```
- value
```

```
- unit
```

```
- reference_range
```

```
"""
```

Store in SQLite

6. 🧠 FAISS Vector Store Setup

Create Index

```
from langchain.vectorstores import FAISS
```

```
from langchain.embeddings import OpenAIEmbeddings
```

```
embeddings = OpenAIEmbeddings()
```

```
vector_store = FAISS.from_texts(
```

```
    [text],
```

```
    embedding=embeddings
```

```
)
```

```
vector_store.save_local("data/faiss_index")
```

Load Index

```
vector_store = FAISS.load_local("data/faiss_index", embeddings)
```

7. 🤖 Agent Design (Core Implementation)

Agent 1: Intake Agent

Input

- Age
- Gender
- Height
- Weight
- Symptoms

Output

```
{  
  "bmi": 28,  
  "category": "Overweight"  
}
```

Agent 2: Document Analysis Agent

- Input: extracted text
 - Output: structured parameters
-

Agent 3: Health Risk Agent

Logic

```
if value > max_range:
    risk = "High"
elif value < min_range:
    risk = "Low"
```

Agent 4: Disease Pattern Detection

Use hybrid approach:

```
if glucose > 110 and triglycerides > 150:
    condition = "Metabolic Syndrome"
```

Agent 5: Fitness Plan Generator

Prompt Template

Based on:

- Age
- BMI
- Risk level

Generate:

1. Workout Plan
 2. Diet Plan
 3. Restrictions
 4. Lifestyle Advice
-

8. Drug Interaction Engine (Dev Mode)

Store knowledge as text files

data/drug_knowledge/

Embed into FAISS

Then query:

```
docs = vector_store.similarity_search(user_drugs)
```

9. Chat System (Core Feature)

Flow

User Query → LangChain → Retrieve Context → LLM → Response

RAG Setup

```
retriever = vector_store.as_retriever()
```

```
qa_chain = RetrievalQA.from_chain_type(  
    llm=llm,  
    retriever=retriever  
)
```

10. Dev Mode Security (Basic)

Even in dev:

- Use user_id isolation
- Store separate FAISS index per user

```
data/faiss_index/{user_id}/
```

11. Risk Scoring Engine

```
score = 0
```



```
if glucose > 110:
```

```
    score += 30
```

```
if bmi > 30:
```

```
    score += 20
```

```
if score > 70:
```

```
    risk = "High"
```

12. 🏋️ Fitness Plan Logic

Classification

```
if bmi < 25:
```

```
    level = "Intermediate"
```

```
elif bmi > 30:
```

```
    level = "Beginner"
```

Output Structure

```
{  
    "workout": ["Walking", "Light Strength"],  
    "diet": ["High Protein", "Low Sugar"],  
    "restrictions": ["Avoid heavy lifting"],  
    "lifestyle": ["Sleep 7 hours"]  
}
```

13. 🌐 API Endpoints (FastAPI)

Upload PDF

POST /upload

Ask Question

POST /chat

Get Insights

GET /insights/{user_id}

Generate Plan

POST /fitness-plan

14. 🧪 Testing Strategy

Unit Tests

- PDF parsing
- Risk scoring

Integration Tests

- Full pipeline:

PDF → Insights → Plan

15. ⚠️ Guardrails (MANDATORY)

Add to every response:

- No diagnosis
- Suggest doctor for high risk

Prompt Safety

Do NOT provide medical diagnosis.

Provide only risk-based insights.

16. 🚀 Dev Milestones

Phase 1 (Day 1–3)

- Setup backend + SQLite
- PDF ingestion working

Phase 2 (Day 4–6)

- FAISS + RAG
- Chat system

Phase 3 (Day 7–10)

- Risk engine
- Fitness plan generator

Phase 4

- UI + WhatsApp integration
-

💡 Final Developer Advice

This dev-mode system should behave like:

- **Single-user safe**
 - **Stateless but structured**
 - **Fully testable locally**
-

If you want next step, I can:

- Generate **complete FastAPI code** (ready-to-run)
- Build **LangGraph agent workflow** (visual + code)
- Create **prompt library for all agents** (production-grade)